Less JavaScript, More Platform

Modern Browser APIs Front-End Developers Should Actually Use

TechBash 2026

Front End / Web

Courtney Yatteau
Developer Advocate, Esri



The browser changed. Our defaults didn’t.

Many teams still reach for JavaScript before checking what the platform already gives them.

Need a modal?
Add JavaScript.

Need a floating panel?
Add JavaScript.

Need form validation?
Add JavaScript.

Need a smoother state change?
Add JavaScript.

But do we still need to?

Important: this is not an anti-JavaScript talk

The goal is more intentional JavaScript, not performative minimalism.

JavaScript is great at application logic

JavaScript is less exciting when it recreates native behavior

The best UI often uses both: platform first, JS on purpose

Keep some JS. We question the plumbing around it.

The hidden cost of 'just a little JS'

Implementation

Open and close UI
Wire event handlers

Interaction

Keyboard + focus
Dismiss and restore

Maintenance

Browser testing
Accessibility checks
More failure modes

> **Small custom behaviors can become long-lived systems.**

Before another event listener, PAUSE

Do I actually need JavaScript for this?

- **P** **Pause** before reaching for JavaScript
- **A** **Ask** what HTML and CSS can already do
- **U** **Use** the browser platform
- **S** **Study** support and fallbacks
- **E** **Employ** JavaScript where it adds value

Platform first. JavaScript on purpose.

PAUSE in action: a simple FAQ

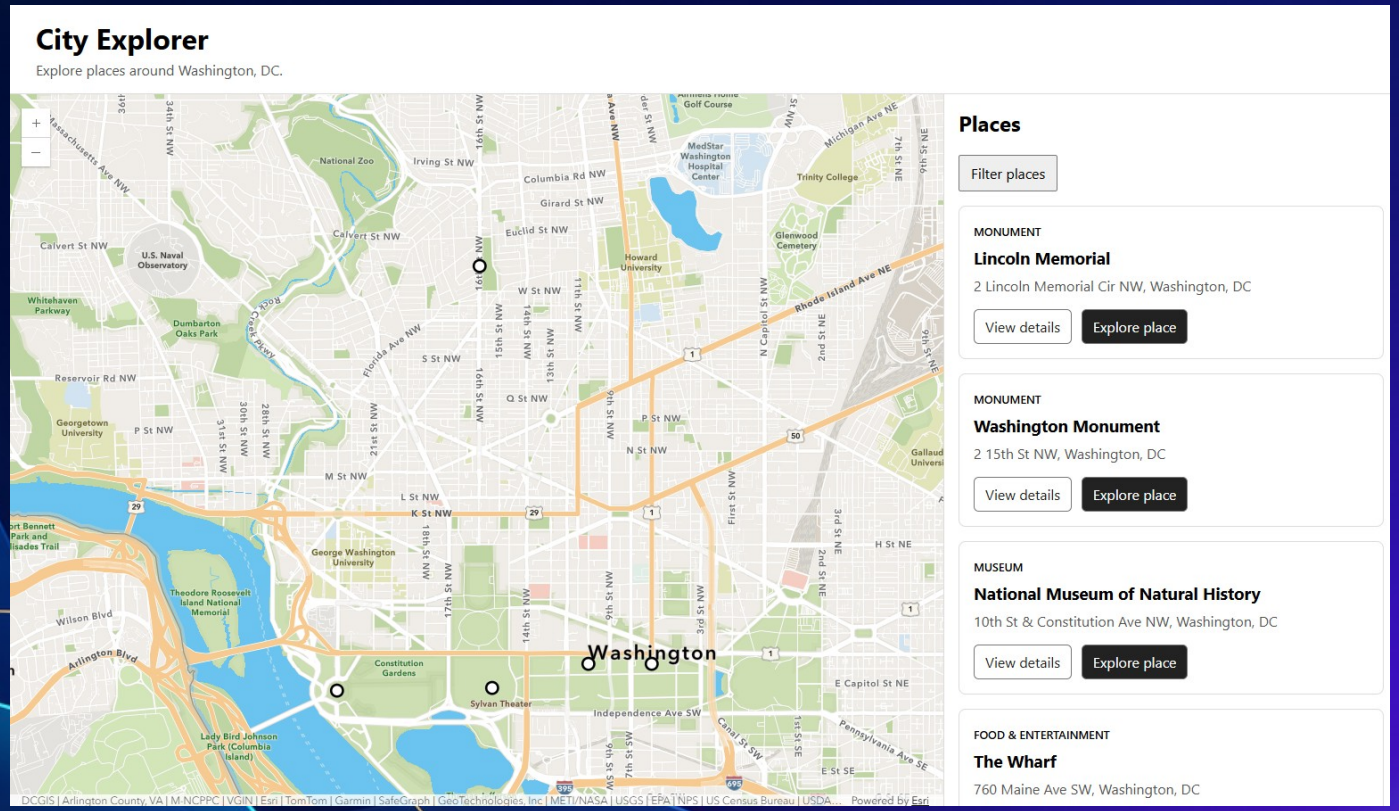
Request: click a question to reveal its answer.

- **P** **Pause** before writing a click listener
- **A** **Ask** if HTML can show and hide content
- **U** **Use** <details> and <summary>
- **S** **Study** support and keyboard behavior
- **E** **Employ** JS only if answers need live data

Sometimes the best JS is the code we don’t need.

Meet City Explorer

A small DC landmarks app for exploring what JavaScript should handle.



ArcGIS map
place cards
dialog details
category filter
state transition

The map stays. The plumbing changes.

Our demo path

Slides explain the code. Browser demos show the behavior.

1	**Modal details**	<dialog> + Commands
2	**Filter panel**	Popover API
3	**Registration form**	Native validation
4	**Selected place**	View Transitions
5	**Wrap-up**	Decision framework

Our shared thread: PAUSE before each demo. Platform first. JavaScript on purpose.

Demo 1: the hand-rolled modal

FIRST: SEE THE CUSTOM MODAL IN CHROME

01 OPEN

```
modalTitle.textContent = place.name;
backdrop.hidden = false; modal.hidden = false;
```

02 CLOSE

```
closeButton.addEventListener("click", closePlaceModal);
backdrop.addEventListener("click", closePlaceModal);
```

03 ESCAPE

```
if (event.key === "Escape" && !modal.hidden) {
  closePlaceModal();
}
```

04 FOCUS

```
lastFocusedElement = document.activeElement;
closeButton.focus();  // on open
lastFocusedElement?.focus();  // on close
```

It works. But we own all of this.

Actual code excerpts from demos/city-explorer/00-start.html

What are we really maintaining?

Backdrop
visual layer

Escape
keyboard dismissal

Focus
where does focus go?

Inertness
can the page behind be used?

State
which place is selected?

Only one of these is truly app-specific.

First move: native `<dialog>`

Move the modal primitive back to the browser.

```
<dialog id="place-dialog">
  <h2>Lincoln Memorial</h2>
  <button>Close</button>
</dialog>

dialog.showModal();
```

Browser owns:

- **top layer**
- **backdrop**
- **inert page behind**
- **Escape behavior**
- **dialog semantics**

Source note: MDN describes showModal() as widely available since March 2022; modal dialogs enter the top layer, receive ::backdrop, and make the document behind inert.

2026 move: Invoker Commands

The “open the dialog” wiring can become declarative too.

```
<button
  command="show-modal"
  commandfor="place-dialog">
  View details
</button>

<dialog id="place-dialog">
  ...
  <button command="close" commandfor="place-dialog">
    Close
  </button>
</dialog>
```

No listener just to call showModal().

HTML opens and closes. JavaScript populates the selected place.

Source note: MDN lists Invoker Commands as Baseline 2025, newly available since December 2025.

What changed?

PAUSE

Before

JS opens
JS closes
JS handles Escape
JS handles backdrop
JS manages focus

After

HTML invokes <dialog> owns modal behavior
JS fills in selected place
JS remains useful

Takeaway

JavaScript should manage application state. It does not also need to reinvent the dialog.

PAUSE CHECK · Dialog: HTML owns modal behavior | JS owns selected-place details

Native is a start, not a testing shortcut

PAUSE

Browser gives

Modal top layer
Escape behavior
Background inertness

You still check

Dialog labeling
Focus entry + return
Keyboard flow

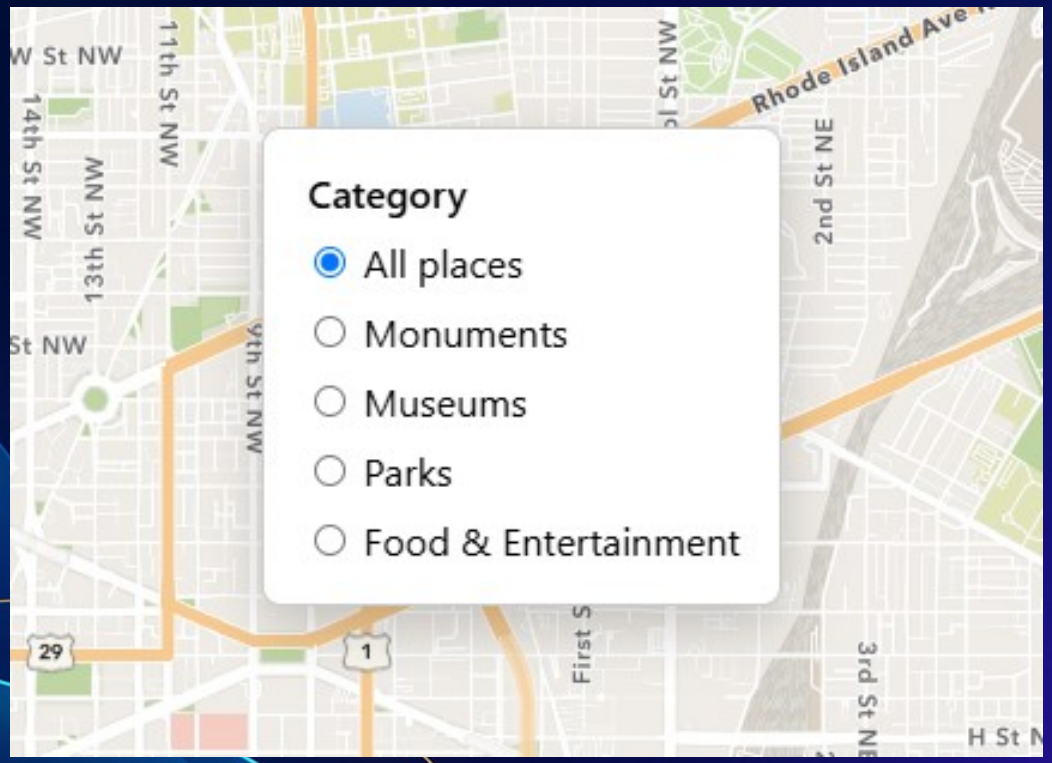
App still owns

Selected-place data
Content changes
Recovery behavior

PAUSE CHECK · Less plumbing does not mean less testing.

Demo 2: the floating filter panel

FIRST: SEE THE FILTER PANEL IN CHROME



Filter places
All · Monuments · Museums · Parks · Food

Opening a panel is generic.
Filtering places is app logic.

Before: custom floating UI

```
filterButton.addEventListener("click", togglePanel);
document.addEventListener("click", maybeClosePanel);
document.addEventListener("keydown", handleEscape);

function applyFilter(category) {
  // real app logic
}
```

One file now owns two jobs:

1. **generic popover behavior**
2. **actual filtering behavior**

After: Popover API

```
<button popovertarget="filter-popover">
  Filter places
</button>

<div id="filter-popover" popover>
  ...radio buttons...
</div>
```

Browser handles

- Opening and closing
- Top-layer placement
- Click-outside dismissal
- Escape dismissal

JavaScript still filters the map and cards.

Source: MDN, Popover API (auto popovers support light dismiss and Escape).

The useful split

PAUSE

Platform behavior

Open panel
Close panel
Dismiss outside
Dismiss on Escape

Application behavior

Which category?
Which cards?
Which graphics?
What app state?

That split is the mental model.

PAUSE CHECK · Popover: browser opens the panel | JS filters the map and cards

Demo 3: forms are the oldest new lesson

FIRST: TRY THE CUSTOM FORM IN CHROME

```
if (name.value.trim() === "") { ... }
if (!email.value.includes("@")) { ... }
if (attendees < 1 ) { ... }

showErrorMessage();
blockSubmit();
```

Some features are not new.
They are just still underused.

Move rules into the markup

```
<input name="name" required>

<input name="email" type="email" required>

<input
  name="attendees"
  type="number"
  min="1"
  max="10"
  required
>
```

HTML now owns:
- **required fields**
- **email format**
- **min / max**
- **blocking invalid submit**

JavaScript handles what happens after valid submit.

Forms takeaway

PAUSE

> **Let HTML do HTML’s job first.**

Then add JavaScript for:

- custom submission
- async validation
- server response
- multi-step workflows
- genuinely custom rules

PAUSE CHECK · Forms: HTML validates constraints | JS handles submission

Native form validation has boundaries

Browser
Required, type, min/max

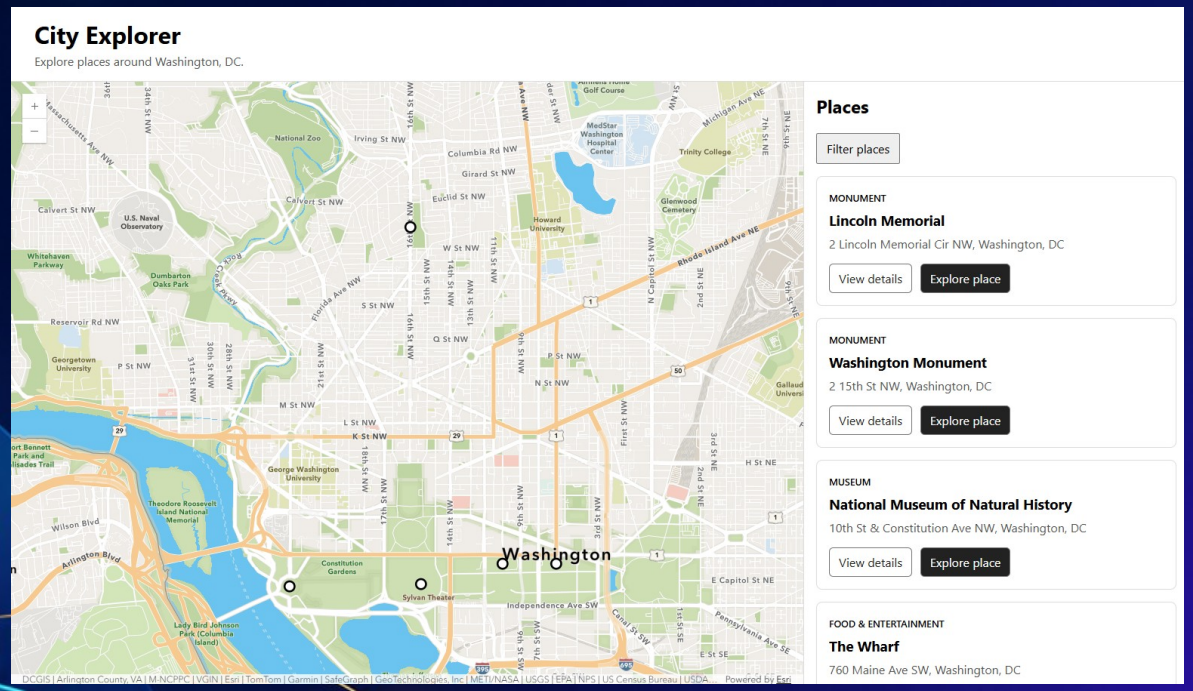
JavaScript
Async or cross-field rules

Server
Verify, authorize, save

Client-side validation improves UX; it is not a trust boundary.

Demo 4: state changes can feel abrupt

FIRST: SEE THE ABRUPT SWITCH IN CHROME




```
selectedTitle.textContent =
place.name;
selectedCategory.textContent =
place.category;
selectedAddress.textContent =
place.address;

placeList.hidden = true;
selectedPlace.hidden = false;
```

The app works. The change feels sudden.

View Transitions: keep the state change

```
function transitionTo(updateDOM) {
  if (!document.startViewTransition) {
    updateDOM();
    return;
  }

  document.startViewTransition(updateDOM);
}

transitionTo(() => {
  placeList.hidden = true;
  selectedPlace.hidden = false;
});
```

JavaScript still changes state.

The browser handles the visual transition between states.

Source note: MDN lists startViewTransition() / ViewTransition as Baseline 2025, newly available since October 2025.

This is not less capable

PAUSE

> **Less JavaScript does not mean less polished.**

With View Transitions:

- same app logic
- fewer custom animation systems
- smoother perceived state changes
- reasonable fallback path

PAUSE CHECK · Transitions: browser animates | JS changes state

How to judge a platform enhancement

Functionality

Does the state change work without the effect?

User preference

Respect reduced motion
Keep focus predictable

Delivery

Check support
Keep a fallback
Test real browsers

> A smoother transition still has to be a usable transition.

Progressive enhancement is the thread

Core behavior
Works first

Platform upgrade
Use native capability

Fallback path
Keep the app usable

> **Adoption strategy matters as much as the API.**

Rapid-fire: small features, real savings

<details> &
<summary>
Basic disclosure UI

loading="lazy"
Lazy-load images

inert
Make a region non-interactive

Browser support: do not guess

Mature

<dialog> / showModal widely available since 2022

Recent

Popover, View Transitions, Invoker Commands newer adoption decisions

Decision

Check your users. Use fallbacks. Enhance intentionally.

PAUSE: all together

Dialog + commands
Open details with HTML

Popover API
Show and dismiss help

Native validation
Let HTML check form inputs

View Transitions
Animate a view change

One simple page
Platform first. JS on purpose.

Use PAUSE in your next code review

A repeatable browser-first decision, not a rewrite mandate.

- **P** Pause before reaching for JavaScript
- **A** Ask what HTML and CSS can already do
- **U** Use the browser platform
- **S** Study support and fallbacks
- **E** Employ JavaScript where it adds value

Platform first. JavaScript on purpose.

What PAUSE changed

Modal plumbing
→ <dialog> + commands

Floating-panel plumbing
→ Popover API

Form-validation plumbing
→ HTML constraints

Animation plumbing
→ View Transitions

Guessing support
→ deliberate adoption

You do not need a rewrite

> **Start with one interaction.**

- Pick a modal
- Pick a form
- Pick a menu
- Pick a transition

Replace one custom system with one native primitive. Learn from that.

Final thought

> The goal was never to make the app JavaScript-free.

> The goal was to stop spending JavaScript on things the browser already knows how to do.

Before another event listener, PAUSE.

Thank you, TechBash!

Courtney Yatteau

@c_yatteau

courtneyyatteau

@cyatteau.bsky.social

Slides / demos

github.com/cyatteau/techbash-2026-less-javascript-more-platform